

SUMMARY OF CORE C CONCEPTS

- **Preprocessor Directives:** Instructions processed before compilation (e.g., `#include`, `#define`).
- **Header File:** Files containing function declarations and macro definitions (e.g., `stdio.h`, `stdlib.h`).
- **Tokens:** Smallest individual lexical elements of a program (keywords, identifiers, constants, operators).
- **Identifiers & Variables:** User-defined names for variables, functions, and named memory locations.
- **Data Types & Operators:** Data sizes (`int`, `float`, `char`) and operators (arithmetic, relational, logical, bitwise).
- **Control Flow:** Conditional decision making (`if-else`, `switch`) and iterations (`for`, `while`, `do-while`).
- **Arrays & Strings:** Contiguous memory buffers for primitive data and null-terminated character sequences.
- **Pointers & DMA:** Direct memory address manipulation and dynamic allocation (`malloc`, `realloc`, `free`).
- **Structures & Unions:** User-defined composite data structures with custom member fields.

SECTION 1: Variables, Data Types and Operators

Problem 1.1: Reverse Three-Digit Number

Given a three-digit integer, reverse its digits using arithmetic operators only (no loops, arrays, or strings).

```
#include <stdio.h>

int main() {
    int num, rev;
    printf("Enter a 3-digit integer: ");
    if (scanf("%d", &num) != 1) return 1;

    int d1 = num % 10;           // Extract last digit
    int d2 = (num / 10) % 10;    // Extract middle digit
    int d3 = num / 100;         // Extract first digit

    rev = d1 * 100 + d2 * 10 + d3;
    printf("Output: %d\n", rev);
    return 0;
}
```

EXPLANATION & REASONING: We use the modulo operator (`% 10`) to extract individual units from the three-digit number and integer division (`/ 10`) to shift right. Recombining them as `d1*100 + d2*10 + d3` effectively reverses the digits without any looping constructs.

STEP-BY-STEP DRY RUN: Input = 456. 1. $d1 = 456 \% 10 = 6$. 2. $d2 = (456 / 10) \% 10 = 45 \% 10 = 5$. 3. $d3 = 456 / 100 = 4$. 4. $rev = 6 * 100 + 5 * 10 + 4 = 600 + 50 + 4 = 654$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.2: Electricity Bill Calculator

Calculate total electricity bill based on slabs: First 100 units @ ₹3/unit, next 100 units @ ₹5/unit, remaining @ ₹8/unit.

```
#include <stdio.h>

int main() {
    int units;
    float bill = 0;
    printf("Enter units: ");
    if (scanf("%d", &units) != 1) return 1;

    if (units <= 100) {
        bill = units * 3;
    } else if (units <= 200) {
        bill = 100 * 3 + (units - 100) * 5;
    } else {
        bill = 100 * 3 + 100 * 5 + (units - 200) * 8;
    }
    printf("Output: %.0f\n", bill);
    return 0;
}
```

EXPLANATION & REASONING: The bill is computed using conditional branching based on consumption slabs. Units exceeding 200 pay full rate for the first two slabs ($100 \times 3 + 100 \times 5$) plus ₹8 per remaining unit.

STEP-BY-STEP DRY RUN: Input = 250. 1. units (250) > 200 -> Branch to else. 2. bill = $(100 \times 3) + (100 \times 5) + ((250 - 200) \times 8)$ 3. bill = $300 + 500 + (50 \times 8) = 800 + 400 = 1100$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.3: Time Conversion

Convert total input seconds into Hours, Minutes, and Seconds.

```
#include <stdio.h>

int main() {
    int total_sec, hrs, min, sec;
    printf("Enter total seconds: ");
    if (scanf("%d", &total_sec) != 1) return 1;

    hrs = total_sec / 3600;
    min = (total_sec % 3600) / 60;
    sec = total_sec % 60;

    printf("%d Hours %d Minutes %d Seconds\n", hrs, min, sec);
    return 0;
}
```

EXPLANATION & REASONING: 1 Hour = 3600 seconds. Dividing total seconds by 3600 gives total hours. The remainder modulo 3600 gives total remaining seconds, which when divided by 60 gives minutes, and modulo 60 gives remaining seconds.

STEP-BY-STEP DRY RUN: Input = 7384. 1. hrs = $7384 / 3600 = 2$. 2. Remainder = $7384 \% 3600 = 184$. 3. min = $184 / 60 = 3$. 4. sec = $184 \% 60 = 4$. Output = '2 Hours 3 Minutes 4 Seconds'.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.4: Salary Calculation

Calculate Gross Salary where HRA = 20% of Basic, DA = 40% of Basic.

```
#include <stdio.h>

int main() {
    float basic, hra, da, gross;
    printf("Enter Basic Salary: ");
    if (scanf("%f", &basic) != 1) return 1;

    hra = 0.20f * basic;
    da = 0.40f * basic;
    gross = basic + hra + da;

    printf("Output: %.2f\n", gross);
    return 0;
}
```

EXPLANATION & REASONING: Gross salary is calculated as the sum of Basic salary, House Rent Allowance (HRA), and Dearness Allowance (DA). HRA is $0.2 \times \text{Basic}$, and DA is $0.4 \times \text{Basic}$, totaling $1.6 \times \text{Basic}$.

STEP-BY-STEP DRY RUN: Input basic = 25000. 1. hra = $0.20 \times 25000 = 5000$. 2. da = $0.40 \times 25000 = 10000$. 3. gross = $25000 + 5000 + 10000 = 40000.00$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.5: Swap Without Temporary Variable

Swap two integers using arithmetic operators only without creating a auxiliary variable.

```
#include <stdio.h>

int main() {
    int a, b;
    printf("Enter two numbers: ");
    if (scanf("%d %d", &a, &b) != 2) return 1;

    a = a + b;
    b = a - b;
    a = a - b;

    printf("Output: %d %d\n", a, b);
    return 0;
}
```

EXPLANATION & REASONING: Summing both variables ($a = a + b$) holds their combined value. Subtracting b from this sum yields original a (stored in b), and subtracting new b from sum yields original b (stored in a).

STEP-BY-STEP DRY RUN: Input a = 15, b = 25. 1. $a = 15 + 25 = 40$. 2. $b = 40 - 25 = 15$ (original a). 3. $a = 40 - 15 = 25$ (original b). Result: a = 25, b = 15.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.6: Find Largest Without Relational Operators

Find the maximum of two integers without using `if`, `switch`, loops, relational operators (>, <), or ternary operators.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int a, b;
    printf("Enter two numbers: ");
    if (scanf("%d %d", &a, &b) != 2) return 1;

    int max = (a + b + abs(a - b)) / 2;
    printf("Output: %d\n", max);
    return 0;
}
```

EXPLANATION & REASONING: The mathematical formula $\text{max}(a,b) = (a + b + |a - b|) / 2$ utilizes the absolute difference between `a` and `b`. If $a \geq b$, $a - b$ is positive, giving $2a/2 = a$. If $a < b$, $|a - b|$ equals $b - a$, giving $2b/2 = b$.

STEP-BY-STEP DRY RUN: $a = 45, b = 78$. 1. $|a - b| = |45 - 78| = |-33| = 33$. 2. $\text{max} = (45 + 78 + 33) / 2 = 156 / 2 = 78$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.7: Digital Lock Code

Given a 4-digit number, add 5 to each digit, take modulo 10 of each, and reverse the final digit sequence.

```
#include <stdio.h>

int main() {
    int code;
    printf("Enter a 4-digit number: ");
    if (scanf("%d", &code) != 1) return 1;

    int d1 = (code / 1000 + 5) % 10;
    int d2 = ((code / 100) % 10 + 5) % 10;
    int d3 = ((code / 10) % 10 + 5) % 10;
    int d4 = (code % 10 + 5) % 10;

    // Reverse digit order: d4 becomes thousands, d3 hundreds, d2 tens, d1 units
    int result = d4 * 1000 + d3 * 100 + d2 * 10 + d1;
    printf("Output: %d\n", result);
    return 0;
}
```

EXPLANATION & REASONING: Each digit from thousands to units is isolated using integer division and modulo arithmetic. We add 5 and apply `% 10` to wrap around, then construct the reversed number.

STEP-BY-STEP DRY RUN: Input = 1234. 1. $d1 = (1 + 5) \% 10 = 6$. 2. $d2 = (2 + 5) \% 10 = 7$. 3. $d3 = (3 + 5) \% 10 = 8$. 4. $d4 = (4 + 5) \% 10 = 9$. 5. Reversed = $9 * 1000 + 8 * 100 + 7 * 10 + 6 = 9876$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 1.8: Currency Note Calculator

Find the minimum number of currency notes required for ₹500, 200, 100, 50, 20, 10, 5, 2, 1.

```
#include <stdio.h>

int main() {
    int amount;
    int notes[] = {500, 200, 100, 50, 20, 10, 5, 2, 1};
    printf("Enter amount: ");
    if (scanf("%d", &amount) != 1) return 1;

    for (int i = 0; i < 9; i++) {
        int count = amount / notes[i];
        amount %= notes[i];
        printf("%d:%d%s", notes[i], count, (i == 8) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: A greedy approach processes notes from highest to lowest denomination. Integer division determines how many notes of that denomination fit, and the modulo operator updates the remaining amount.

STEP-BY-STEP DRY RUN: Amount = 2893. - 500: $2893 / 500 = 5$, rem = 393. - 200: $393 / 200 = 1$, rem = 193. - 100: $193 / 100 = 1$, rem = 93. - 50: $93 / 50 = 1$, rem = 43. - 20: $43 / 20 = 2$, rem = 3. - 10: $3 / 10 = 0$, rem = 3. - 5: $3 / 5 = 0$, rem = 3. - 2: $3 / 2 = 1$, rem = 1. - 1: $1 / 1 = 1$, rem = 0.

Time Complexity: O(1) - fixed array of 9 denominations

Space Complexity: O(1)

SECTION 2: Conditional Statements and Looping Statements

Problem 2.1: Equality Check

Accept two integers and check whether they are equal.

```
#include <stdio.h>

int main() {
    int num1, num2;
    if (scanf("%d %d", &num1, &num2) != 2) return 1;
    if (num1 == num2)
        printf("Number1 and Number2 are equal\n");
    else
        printf("Number1 and Number2 are not equal\n");
    return 0;
}
```

EXPLANATION & REASONING: Uses the equality operator (`==`) inside an `if-else` statement to compare two integer inputs.

STEP-BY-STEP DRY RUN: Inputs: num1 = 15, num2 = 15. Condition (15 == 15) is True. Print 'Number1 and Number2 are equal'.

Time Complexity: O(1) **Space Complexity:** O(1)

Problem 2.2: Even or Odd Check

Check whether a given integer is even or odd.

```
#include <stdio.h>

int main() {
    int num;
    if (scanf("%d", &num) != 1) return 1;
    if (num % 2 == 0)
        printf("%d is an even integer\n", num);
    else
        printf("%d is an odd integer\n", num);
    return 0;
}
```

EXPLANATION & REASONING: An integer is even if its remainder when divided by 2 (`num % 2`) is 0. Otherwise, it is odd.

STEP-BY-STEP DRY RUN: Input = 15. 15 % 2 = 1 != 0. Evaluates to false -> branch to else. Output: '15 is an odd integer'.

Time Complexity: O(1) **Space Complexity:** O(1)

Problem 2.3: Positive or Negative Check

Check whether a given integer is positive or negative.

```
#include <stdio.h>

int main() {
    int num;
    if (scanf("%d", &num) != 1) return 1;
    if (num >= 0)
        printf("%d is a positive number\n", num);
    else
        printf("%d is a negative number\n", num);
    return 0;
}
```

EXPLANATION & REASONING: Compares input number with 0 using `>=`. Non-negative numbers are printed as positive.

STEP-BY-STEP DRY RUN: Input = 15. 15 >= 0 is True. Output: '15 is a positive number'.

Time Complexity: O(1) **Space Complexity:** O(1)

Problem 2.4: Leap Year Determination

Determine if a given year is a leap year.

```
#include <stdio.h>

int main() {
    int year;
    if (scanf("%d", &year) != 1) return 1;
    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
        printf("%d is a leap year.\n", year);
    else
        printf("%d is not a leap year.\n", year);
    return 0;
}
```

EXPLANATION & REASONING: A year is a leap year if it is divisible by 400 OR divisible by 4 but not by 100.

STEP-BY-STEP DRY RUN: Input year = 2016. $2016 \% 4 == 0$ (True) and $2016 \% 100 != 0$ (True). Entire condition evaluates to True. Output: '2016 is a leap year.'

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 2.5: Voting Eligibility

Read age of a candidate and determine whether they are eligible to cast a vote.

```
#include <stdio.h>

int main() {
    int age;
    if (scanf("%d", &age) != 1) return 1;
    if (age >= 18)
        printf("Congratulation! You are eligible for casting your vote.\n");
    else
        printf("You are not eligible to vote.\n");
    return 0;
}
```

EXPLANATION & REASONING: Checks if the candidate's age is greater than or equal to 18.

STEP-BY-STEP DRY RUN: Input age = 21. $21 >= 18$ is True. Output congratulatory message.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 2.6: Signum Function Implementation

Display $n = 1$ when $m > 0$, $n = 0$ when $m = 0$, and $n = -1$ when $m < 0$.

```
#include <stdio.h>

int main() {
    int m, n;
    if (scanf("%d", &m) != 1) return 1;
    if (m > 0) n = 1;
    else if (m == 0) n = 0;
    else n = -1;
    printf("The value of n = %d\n", n);
    return 0;
}
```

EXPLANATION & REASONING: Executes nested 'if-else' branches to map an integer 'm' to sign value 'n' (-1, 0, or 1).

STEP-BY-STEP DRY RUN: Input m = -5. $m > 0$ (False), $m == 0$ (False). Else branch executes $\rightarrow n = -1$. Output: 'The value of n = -1'.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 2.7: Height Categorization

Categorize a person's height in centimeters ($<150\text{cm} = \text{Dwarf}$).

```
#include <stdio.h>

int main() {
    float height;
    if (scanf("%f", &height) != 1) return 1;
    if (height < 150.0f)
        printf("The person is Dwarf.\n");
    else if (height <= 165.0f)
        printf("The person is Average Height.\n");
    else
        printf("The person is Tall.\n");
    return 0;
}
```

EXPLANATION & REASONING: Categorizes floating point height values into designated height classes.

STEP-BY-STEP DRY RUN: Input = 135.0. $135.0 < 150.0$ is True. Output: 'The person is Dwarf.'

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 2.8: Largest of Three Numbers

Find the maximum among three numbers.

```
#include <stdio.h>

int main() {
    int n1, n2, n3;
    if (scanf("%d %d %d", &n1, &n2, &n3) != 3) return 1;
    printf("1st Number = %d, 2nd Number = %d, 3rd Number = %d\n", n1, n2, n3);
    if (n1 >= n2 && n1 >= n3)
        printf("The 1st Number is the greatest among three\n");
    else if (n2 >= n1 && n2 >= n3)
        printf("The 2nd Number is the greatest among three\n");
    else
        printf("The 3rd Number is the greatest among three\n");
    return 0;
}
```

EXPLANATION & REASONING: Uses logical AND (`&&`) in conditional statements to compare each variable against the other two.

STEP-BY-STEP DRY RUN: $n1=12, n2=25, n3=52$. - $(12 \geq 25 \ \&\& \ 12 \geq 52) \rightarrow \text{False}$. - $(25 \geq 12 \ \&\& \ 25 \geq 52) \rightarrow \text{False}$. - Else executes: 'The 3rd Number is the greatest among three'.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 2.9: Coordinate Quadrant Identification

Determine in which quadrant an (X, Y) coordinate point lies.

```
#include <stdio.h>

int main() {
    int x, y;
    if (scanf("%d %d", &x, &y) != 2) return 1;
    if (x > 0 && y > 0)
        printf("The coordinate point (%d,%d) lies in the First quadrant.\n", x, y);
    else if (x < 0 && y > 0)
        printf("The coordinate point (%d,%d) lies in the Second quadrant.\n", x, y);
    else if (x < 0 && y < 0)
        printf("The coordinate point (%d,%d) lies in the Third quadrant.\n", x, y);
    else if (x > 0 && y < 0)
        printf("The coordinate point (%d,%d) lies in the Fourth quadrant.\n", x, y);
    else
        printf("The coordinate point (%d,%d) lies at the Origin or on an Axis.\n", x, y);
    return 0;
}
```

EXPLANATION & REASONING: Checks sign combinations of coordinates x and y to assign quadrant 1 (+,+), 2 (-,+), 3 (-,-), or 4 (+,-).

STEP-BY-STEP DRY RUN: x=7, y=9. x > 0 and y > 0 are both True. Output: First quadrant.

Time Complexity: O(1) **Space Complexity:** O(1)

Problem 2.10: Vowel or Consonant Check

Check whether an input character is a vowel or a consonant.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch;
    if (scanf(" %c", &ch) != 1) return 1;
    char lower = tolower(ch);
    if (lower == 'a' || lower == 'e' || lower == 'i' || lower == 'o' || lower == 'u')
        printf("The alphabet is a vowel\n");
    else
        printf("The alphabet is a consonant\n");
    return 0;
}
```

EXPLANATION & REASONING: Normalizes char to lowercase and compares against vowel list ('a','e','i','o','u').

STEP-BY-STEP DRY RUN: Input = 'k'. Lowercase = 'k'. Matches none of 'a,e,i,o,u'. Outputs: 'The alphabet is a consonant'.

Time Complexity: O(1) **Space Complexity:** O(1)

SECTION 3: Arrays

Problem 3.1: Array Store & Print

Store 10 elements in an array and print them.

```
#include <stdio.h>

int main() {
    int arr[10];
    for (int i = 0; i < 10; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
    }
    printf("Elements in array are: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Reads 10 integer inputs sequentially into a contiguous fixed-size integer array using a for loop, then prints them in order.

STEP-BY-STEP DRY RUN: i=0..9 reads [1, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Second loop traverses indices 0 to 9 printing each element separated by space.

Time Complexity: O(N) where N = 10

Space Complexity: O(N)

Problem 3.2: Array Reverse Display

Read N values in an array and display them in reverse order.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
    }
    for (int i = n - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Loads N elements into a Variable Length Array (VLA) and iterates backward from index `n - 1` down to 0.

STEP-BY-STEP DRY RUN: n=3, arr = [2, 5, 7]. Loop i from 2 down to 0: arr[2]=7, arr[1]=5, arr[0]=2.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 3.3: Array Sum Calculation

Find the sum of all elements in an array.

```
#include <stdio.h>

int main() {
    int n, sum = 0;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
        sum += arr[i];
    }
    printf("Sum of all elements stored in the array is : %d\n", sum);
    return 0;
}
```

EXPLANATION & REASONING: Maintains a running sum accumulator variable initialized to 0 and adds each array element during input traversal.

STEP-BY-STEP DRY RUN: n=3, elements: 2, 5, 8. sum starts at 0. sum = 0 + 2 = 2; sum = 2 + 5 = 7; sum = 7 + 8 = 15.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 3.4: Array Copy

Copy the elements of one array into another array.

```
#include <stdio.h>

int main() {
    int n = 3;
    int source[3], dest[3];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &source[i]) != 1) return 1;
        dest[i] = source[i];
    }
    for (int i = 0; i < n; i++) printf("%d ", dest[i]);
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Iterates through the source array and assigns `dest[i] = source[i]` element-by-element.

STEP-BY-STEP DRY RUN: source = [15, 10, 12]. dest[0] = 15, dest[1] = 10, dest[2] = 12. Prints contents of dest array.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 3.5: Count Duplicate Elements

Count the total number of duplicate elements in an array.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n], visited[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
        visited[i] = 0;
    }
    int dupes = 0;
    for (int i = 0; i < n; i++) {
        if (visited[i]) continue;
        int count = 1;
        for (int j = i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
                visited[j] = 1;
            }
        }
        if (count > 1) dupes++;
    }
    printf("Total number of duplicate elements found in the array is : %d\n", dupes);
    return 0;
}
```

EXPLANATION & REASONING: Uses a boolean `visited` array to keep track of elements already processed. A nested loop counts occurrences of unvisited elements.

STEP-BY-STEP DRY RUN: arr = [5, 1, 1]. i=0 (val 5): no duplicate. i=1 (val 1): matches arr[2] (val 1), visited[2] set to 1, dupes count becomes 1.

Time Complexity: $O(N^2)$

Space Complexity: $O(N)$

Problem 3.6: Print Unique Elements

Print all unique elements (elements appearing exactly once) in an array.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
    }
    printf("The unique elements found in the array are: ");
    for (int i = 0; i < n; i++) {
        int isUnique = 1;
        for (int j = 0; j < n; j++) {
            if (i != j && arr[i] == arr[j]) {
                isUnique = 0;
                break;
            }
        }
        if (isUnique) printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: For each element at index `i`, we check if it appears anywhere else (`j != i`). If no duplicate matches, `isUnique` stays 1 and the element is printed.

STEP-BY-STEP DRY RUN: arr = [3, 2, 2, 5]. - i=0 (3): no other matches -> print 3. - i=1 (2): matches arr[2] -> isUnique=0. - i=2 (2): matches arr[1] -> isUnique=0. - i=3 (5): no other matches -> print 5.

Time Complexity: $O(N^2)$

Space Complexity: $O(N)$

Problem 3.7: Merge Two Sorted Arrays (Descending)

Merge two sorted arrays of the same size into a single array sorted in descending order.

```
#include <stdio.h>

int main() {
    int a[] = {3, 2, 1}, b[] = {3, 2, 1};
    int n = 3, merged[6];
    int i = 0, j = 0, k = 0;

    while (i < n && j < n) {
        if (a[i] >= b[j]) merged[k++] = a[i++];
        else merged[k++] = b[j++];
    }
    while (i < n) merged[k++] = a[i++];
    while (j < n) merged[k++] = b[j++];

    for (int idx = 0; idx < 6; idx++) printf("%d ", merged[idx]);
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Two-pointer technique compares current elements from both descending input arrays and copies the larger element to the merged array.

STEP-BY-STEP DRY RUN: a=[3,2,1], b=[3,2,1]. - Compare 3 vs 3 -> take a[0]=3. - Compare 2 vs 3 -> take b[0]=3. - Compare 2 vs 2 -> take a[1]=2. - Compare 1 vs 2 -> take b[1]=2. - Compare 1 vs 1 -> take a[2]=1, then b[2]=1. Merged = [3, 3, 2, 2, 1, 1].

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 3.8: Frequency of Array Elements

Count the frequency of each element of an array.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n], visited[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
        visited[i] = 0;
    }
    for (int i = 0; i < n; i++) {
        if (visited[i]) continue;
        int count = 1;
        for (int j = i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
                visited[j] = 1;
            }
        }
        printf("%d occurs %d times ", arr[i], count);
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Iterates through the array and marks duplicate instances as visited so each distinct value's total frequency is reported only once.

STEP-BY-STEP DRY RUN: arr=[25, 12, 43]. None repeat. Loop outputs 25: 1 time, 12: 1 time, 43: 1 time.

Time Complexity: $O(N^2)$

Space Complexity: $O(N)$

Problem 3.9: Find Maximum & Minimum

Find maximum and minimum elements in an array.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n];
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
    }
    int max = arr[0], min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }
    printf("Maximum element is : %d Minimum element is : %d\n", max, min);
    return 0;
}
```

EXPLANATION & REASONING: Initializes `max` and `min` with `arr[0]` and performs a single linear pass through the rest of the array to update boundary values.

STEP-BY-STEP DRY RUN: arr = [45, 25, 21]. Initial max=45, min=45. - i=1 (25): max stays 45, min becomes 25. - i=2 (21): max stays 45, min becomes 21. Result: max=45, min=21.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 3.10: Separate Odd & Even Arrays

Separate odd and even integers into two different arrays.

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int arr[n], even[n], odd[n];
    int eCount = 0, oCount = 0;
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 1;
        if (arr[i] % 2 == 0)
            even[eCount++] = arr[i];
        else
            odd[oCount++] = arr[i];
    }
    printf("The Even elements are : ");
    for (int i = 0; i < eCount; i++) printf("%d ", even[i]);
    printf("The Odd elements are : ");
    for (int i = 0; i < oCount; i++) printf("%d ", odd[i]);
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Evaluates `arr[i] % 2 == 0` during read. Evens are placed into `even[]` array and odds into `odd[]` array.

STEP-BY-STEP DRY RUN: arr = [25, 47, 42, 56, 32]. - 25 % 2 != 0 -> odd[0]=25 - 47 % 2 != 0 -> odd[1]=47 - 42 % 2 == 0 -> even[0]=42 - 56 % 2 == 0 -> even[1]=56 - 32 % 2 == 0 -> even[2]=32.

Time Complexity: O(N)

Space Complexity: O(N)

SECTION 4: Strings

Problem 4.1: String Input Print

Input a string and print it.

```
#include <stdio.h>

int main() {
    char str[100];
    if (fgets(str, sizeof(str), stdin) != NULL) {
        printf("The string you entered is : %s", str);
    }
    return 0;
}
```

EXPLANATION & REASONING: `fgets()` safely reads an entire line including spaces from standard input into character array `str` up to buffer capacity.

STEP-BY-STEP DRY RUN: Input = 'Welcome, w3resource\n'. `fgets()` stores 'Welcome, w3resource\n\0'. `printf` formats and outputs the string.

Time Complexity: $O(N)$ length of string

Space Complexity: $O(N)$

Problem 4.2: String Length Without Library

Find length of a string without using `strlen()`.

```
#include <stdio.h>

int main() {
    char str[100];
    if (scanf("%99s", str) != 1) return 1;
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }
    printf("Length of the string is : %d\n", len);
    return 0;
}
```

EXPLANATION & REASONING: Traverses the character array index by index until encountering the null terminator character (`"\0"`). The index count equals string length.

STEP-BY-STEP DRY RUN: `str` = "w3resource.com". Loop increments `len` from 0 to 14. At index 15, `str[15]` is `"\0"`. Loop ends. Length = 15.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 4.3: Separate String Characters

Separate individual characters from a string with space separation.

```
#include <stdio.h>

int main() {
    char str[100];
    if (fgets(str, sizeof(str), stdin) != NULL) {
        printf("The characters of the string are : ");
        for (int i = 0; str[i] != '\0' && str[i] != '\n'; i++) {
            printf("%c ", str[i]);
        }
        printf("\n");
    }
    return 0;
}
```

EXPLANATION & REASONING: Loops through string indices printing `str[i]` followed by a space until reaching the end-of-line or null character.

STEP-BY-STEP DRY RUN: str = "stellar academy". Loop prints 's', 't', 'e', 'l', 'l', 'a', 'r', ' ', 'a', 'c', 'a', 'd', 'e', 'm', 'y'.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.4: Reverse String Characters

Print individual characters of a string in reverse order.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    if (scanf("%99s", str) != 1) return 1;
    int len = strlen(str);
    printf("The characters of the string in reverse are : ");
    for (int i = len - 1; i >= 0; i--) {
        printf("%c ", str[i]);
    }
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Calculates string length `len`, then iterates backwards from index `len - 1` down to 0, outputting each character with a trailing space.

STEP-BY-STEP DRY RUN: str = "stellar", len = 7. Indices: 6('r'), 5('a'), 4('l'), 3('l'), 2('e'), 1('t'), 0('s'). Output: 'r a l l e t s'.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.5: Count Words in String

Count the total number of words in a sentence string.

```
#include <stdio.h>

int main() {
    char str[200];
    if (fgets(str, sizeof(str), stdin) != NULL) {
        int words = 0, in_word = 0;
        for (int i = 0; str[i] != '\0'; i++) {
            if (str[i] != ' ' && str[i] != '\t' && str[i] != '\n') {
                if (!in_word) {
                    in_word = 1;
                    words++;
                }
            } else {
                in_word = 0;
            }
        }
        printf("Total number of words in the string : %d\n", words);
    }
    return 0;
}
```

EXPLANATION & REASONING: Uses a state flag `in\_word` to detect transitions from whitespace to non-whitespace characters, incrementing the word count on each transition.

STEP-BY-STEP DRY RUN: str = "welcome to stellar\n". - 'w': in_word=0 -> words=1, in_word=1. - ' ': space -> in_word=0. - 't': in_word=0 -> words=2, in_word=1. - ' ': space -> in_word=0. - 's': in_word=0 -> words=3, in_word=1. Total words = 3.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.6: Compare Strings Without Library

Compare two strings character by character without using `strcmp()`.

```
#include <stdio.h>

int compare_strings(char s1[], char s2[]) {
    int i = 0;
    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i]) return 0;
        i++;
    }
    return (s1[i] == '\0' && s2[i] == '\0');
}

int main() {
    char str1[] = "aabbcc", str2[] = "abcdef";
    if (compare_strings(str1, str2))
        printf("Strings are equal.\n");
    else
        printf("Strings are not equal.\n");
    return 0;
}
```

EXPLANATION & REASONING: Traverses both strings simultaneously. If any mismatch is found, returns 0. If both reach null terminator together, returns 1.

STEP-BY-STEP DRY RUN: str1="aabbcc", str2="abcdef". - i=0: 'a'=='a' - i=1: 'a'=='b' -> Mismatch detected! Return 0. Print 'Strings are not equal'.

Time Complexity: O(N) min length of strings

Space Complexity: O(1)

Problem 4.7: Count Alphabets, Digits, Specials

Count total number of alphabets, digits and special characters in a string.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[200];
    if (fgets(str, sizeof(str), stdin) != NULL) {
        int alphabets = 0, digits = 0, specials = 0;
        for (int i = 0; str[i] != '\0' && str[i] != '\n'; i++) {
            if (isalpha((unsigned char)str[i]))
                alphabets++;
            else if (isdigit((unsigned char)str[i]))
                digits++;
            else if (str[i] != ' ')
                specials++;
        }
        printf("Number of Alphabets : %d\nNumber of Digits : %d\nNumber of Special characters : %d\n", alphabets, digits, specials);
    }
    return 0;
}
```

EXPLANATION & REASONING: Uses standard `isalpha()` and `isdigit()` character inspection functions. Characters that are neither letters, digits, nor spaces are counted as special characters.

STEP-BY-STEP DRY RUN: str = "Welcome to stellar123&". - Alphabets ('Welcome to stellar' minus space) = 16 - Digits ('123') = 3 - Specials ('&') = 1.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.8: Copy String

Copy one string to another string manually without `strcpy()`.

```
#include <stdio.h>

int main() {
    char source[200], dest[200];
    int i = 0;
    if (fgets(source, sizeof(source), stdin) != NULL) {
        while (source[i] != '\0' && source[i] != '\n') {
            dest[i] = source[i];
            i++;
        }
        dest[i] = '\0';
        printf("First string : %sSecond string : %s\nNumber of characters copied : %d\n", source, dest, i);
    }
    return 0;
}
```

EXPLANATION & REASONING: Iterates character-by-character from source string to destination string until null/newline is hit, appending null terminator at destination.

STEP-BY-STEP DRY RUN: source = "This is a string to be copied." Loop copies 30 characters into `dest` and sets `dest[30]='\0'`. Total copied = 30.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.9: Count Vowels and Consonants

Count total number of vowels and consonants in a string.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[100];
    if (scanf("%99s", str) != 1) return 1;
    int vowels = 0, consonants = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        char ch = tolower((unsigned char)str[i]);
        if (ch >= 'a' && ch <= 'z') {
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                vowels++;
            else
                consonants++;
        }
    }
    printf("Vowels : %d\nConsonants : %d\n", vowels, consonants);
    return 0;
}
```

EXPLANATION & REASONING: Converts each character to lowercase and checks if it falls in range 'a'-'z'. If it equals 'a','e','i','o','u', vowel count increments, else consonant count increments.

STEP-BY-STEP DRY RUN: str = "Welcome". Lowercase letters: 'w'(C), 'e'(V), 'l'(C), 'c'(C), 'o'(V), 'm'(C), 'e'(V). Vowels=3, Consonants=4.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 4.10: Maximum Frequency Character

Find the character that appears most frequently in a string.

```
#include <stdio.h>

int main() {
    char str[100];
    if (scanf("%99s", str) != 1) return 1;
    int freq[256] = {0};
    for (int i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }
    char max_char = str[0];
    int max_freq = 0;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > max_freq) {
            max_freq = freq[i];
            max_char = (char)i;
        }
    }
    printf("The Highest frequency of character '%c' appears number of times : %d\n", max_char, max_freq);
    return 0;
}
```

EXPLANATION & REASONING: Uses an integer ASCII frequency array of size 256. Increments ASCII index counters during string traversal and tracks the max frequency value.

STEP-BY-STEP DRY RUN: str = "Welcome". Frequency map: 'W':1, 'e':2, 'l':1, 'c':1, 'o':1, 'm':1. Maximum frequency found is 2 for character 'e'.

Time Complexity: O(N)

Space Complexity: O(1) - constant array of size 256

SECTION 5: Structures & Union

Problem 5.1: Student Structure Operations

Create a structure called 'Student' with members name, age, and total marks. Input data for two students, display their info, and find average total marks.

```
#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float marks;
};

int main() {
    struct Student s[2];
    float sum = 0;
    for (int i = 0; i < 2; i++) {
        printf("Enter name, age, marks for student %d: ", i + 1);
        if (scanf("%s %d %f", s[i].name, &s[i].age, &s[i].marks) != 3) return 1;
        sum += s[i].marks;
    }
    for (int i = 0; i < 2; i++) {
        printf("Student %d -> Name: %s, Age: %d, Marks: %.2f\n", i + 1, s[i].name, s[i].age, s[i].marks);
    }
    printf("Average Total Marks = %.2f\n", sum / 2.0f);
    return 0;
}
```

EXPLANATION & REASONING: Declares a `struct Student` containing heterogenous member fields (`name`, `age`, `marks`). Instantiates an array of 2 structures, populates inputs, and calculates the average marks.

STEP-BY-STEP DRY RUN: s[0] = {John, 20, 85.5}, s[1] = {Alice, 21, 92.0}. sum = 85.5 + 92.0 = 177.5. Average = 177.5 / 2 = 88.75.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 5.2: Time Structure Calculations

Define a structure named *Time* with members *hours*, *minutes*, and *seconds*. Input two times, add them, and display in proper time format.

```
#include <stdio.h>

struct Time {
    int hours;
    int minutes;
    int seconds;
};

int main() {
    struct Time t1, t2, res;
    printf("Enter Time 1 (h m s): ");
    if (scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds) != 3) return 1;
    printf("Enter Time 2 (h m s): ");
    if (scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds) != 3) return 1;

    res.seconds = t1.seconds + t2.seconds;
    res.minutes = t1.minutes + t2.minutes + (res.seconds / 60);
    res.seconds %= 60;

    res.hours = t1.hours + t2.hours + (res.minutes / 60);
    res.minutes %= 60;

    printf("%d Hours %d Minutes %d Seconds\n", res.hours, res.minutes, res.seconds);
    return 0;
}
```

EXPLANATION & REASONING: Adds seconds, carrying overflow ($\div 60$) into minutes. Then adds minutes, carrying overflow into hours, resulting in normalized time formatting.

STEP-BY-STEP DRY RUN: $t1 = 2:45:50$, $t2 = 1:20:30$. - sec = $50 + 30 = 80 \rightarrow$ carry $80/60=1$ to min, sec = $80\%60=20$. - min = $45 + 20 + 1 = 66 \rightarrow$ carry $66/60=1$ to hr, min = $66\%60=6$. - hr = $2 + 1 + 1 = 4$. Output: 4 Hours 6 Minutes 20 Seconds.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 5.3: Book Structure Management

Create a structure named *Book* to store details (*title*, *author*, *price*) for 3 books. Find and display the highest and lowest priced books.

```
#include <stdio.h>

struct Book {
    char title[50];
    char author[50];
    float price;
};

int main() {
    struct Book b[3];
    for (int i = 0; i < 3; i++) {
        if (scanf("%s %s %f", b[i].title, b[i].author, &b[i].price) != 3) return 1;
    }
    int highest = 0, lowest = 0;
    for (int i = 1; i < 3; i++) {
        if (b[i].price > b[highest].price) highest = i;
        if (b[i].price < b[lowest].price) lowest = i;
    }
    printf("Most Expensive Book : %s by %s (Price: %.2f)\n", b[highest].title, b[highest].author, b[highest].price);
    printf("Lowest Priced Book : %s by %s (Price: %.2f)\n", b[lowest].title, b[lowest].author, b[lowest].price);
    return 0;
}
```

EXPLANATION & REASONING: Stores 3 book records in an array of structures. Iterates through the array comparing member float field `price` to locate min and max indices.

STEP-BY-STEP DRY RUN: Prices: 350.0, 500.0, 200.0. - $i=1$: $500.0 > 350.0 \rightarrow$ highest = 1. - $i=2$: $200.0 < 350.0 \rightarrow$ lowest = 2. Most expensive: BookB (500.00), Lowest: BookC (200.00).

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 5.4: Circle Structure Calculations

Define a structure named Circle with a radius member. Calculate area and perimeter for two circles.

```
#include <stdio.h>

#define PI 3.14159f

struct Circle {
    float radius;
};

int main() {
    struct Circle c1, c2;
    if (scanf("%f %f", &c1.radius, &c2.radius) != 2) return 1;

    printf("Circle 1 -> Area: %.2f, Perimeter: %.2f\n", PI * c1.radius * c1.radius, 2 * PI * c1.radius);
    printf("Circle 2 -> Area: %.2f, Perimeter: %.2f\n", PI * c2.radius * c2.radius, 2 * PI * c2.radius);
    return 0;
}
```

EXPLANATION & REASONING: Computes circle geometry using formulas $\text{Area} = \pi r^2$ and $\text{Perimeter} = 2\pi r$ using member field `radius`.

STEP-BY-STEP DRY RUN: c1.radius = 5.0. - Area1 = 3.14159 * 25 = 78.54 - Perimeter1 = 2 * 3.14159 * 5 = 31.42.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 5.5: Employee Structure Processing

Create a structure named 'Employee' (id, name, salary) for 3 employees, find the highest salary employee.

```
#include <stdio.h>

struct Employee {
    int id;
    char name[50];
    float salary;
};

int main() {
    struct Employee emp[3];
    for (int i = 0; i < 3; i++) {
        if (scanf("%d %s %f", &emp[i].id, emp[i].name, &emp[i].salary) != 3) return 1;
    }
    int maxIdx = 0;
    for (int i = 1; i < 3; i++) {
        if (emp[i].salary > emp[maxIdx].salary) {
            maxIdx = i;
        }
    }
    printf("Highest Salary Employee: %s (ID: %d, Salary: %.2f)\n", emp[maxIdx].name, emp[maxIdx].id, emp[maxIdx].salary);
    return 0;
}
```

EXPLANATION & REASONING: Reads 3 employee records and finds the index with maximum float value in member field `salary`.

STEP-BY-STEP DRY RUN: Salaries: 50000, 75000, 60000. Index 1 (Bob) has max salary 75000.00.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 5.6: Date Structure Operations

Define a structure named 'Date' with members day, month, and year. Input two dates and find difference in days.

```
#include <stdio.h>
#include <stdlib.h>

struct Date {
    int day;
    int month;
    int year;
};

const int monthDays[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

int countLeapYears(struct Date d) {
    int years = d.year;
    if (d.month <= 2) years--;
    return years / 4 - years / 100 + years / 400;
}

long int getDays(struct Date d) {
    long int n = d.year * 365 + d.day;
    for (int i = 0; i < d.month - 1; i++) n += monthDays[i];
    n += countLeapYears(d);
    return n;
}

int main() {
    struct Date d1, d2;
    if (scanf("%d %d %d", &d1.day, &d1.month, &d1.year) != 3) return 1;
    if (scanf("%d %d %d", &d2.day, &d2.month, &d2.year) != 3) return 1;

    long int days1 = getDays(d1);
    long int days2 = getDays(d2);

    printf("Difference in days : %ld\n", labs(days2 - days1));
    return 0;
}
```

EXPLANATION & REASONING: Converts both dates to total days elapsed since absolute reference point year 0 (including leap year additions) and subtracts them.

STEP-BY-STEP DRY RUN: d1 = 15/8/2023, d2 = 20/8/2023. days2 - days1 = 20 - 15 = 5 days.

Time Complexity: O(1) **Space Complexity:** O(1)

Problem 5.7: Queue Implementation with Structures

Implement a queue structure containing an array, front, and rear indices, with enqueue/dequeue operations.

```
#include <stdio.h>

#define MAX 5

struct Queue {
    int items[MAX];
    int front;
    int rear;
};

void enqueue(struct Queue *q, int value) {
    if (q->rear == MAX - 1) return;
    if (q->front == -1) q->front = 0;
    q->rear++;
    q->items[q->rear] = value;
}

int dequeue(struct Queue *q) {
    if (q->front == -1 || q->front > q->rear) return -1;
    int item = q->items[q->front];
    q->front++;
    return item;
}

int main() {
    struct Queue q = {.front = -1, .rear = -1};
    enqueue(&q, 10);
    enqueue(&q, 20);
    printf("Dequeued: %d\n", dequeue(&q));
    return 0;
}
```

EXPLANATION & REASONING: Implements FIFO (First In First Out) Queue data structure encapsulated inside `struct Queue`. `enqueue` appends to `rear`, `dequeue` retrieves from `front`.

STEP-BY-STEP DRY RUN: Queue init (front=-1, rear=-1). - enqueue(10): front=0, rear=0, items[0]=10. - enqueue(20): rear=1, items[1]=20. - dequeue(): returns items[front=0] -> 10, front becomes 1.

Time Complexity: O(1) per operation

Space Complexity: O(MAX)

Problem 5.8: Complex Number Operations

Create a structure named *Complex* (real, imaginary) to add and multiply two complex numbers.

```
#include <stdio.h>

struct Complex {
    float real;
    float imag;
};

int main() {
    struct Complex c1, c2, sum, prod;
    if (scanf("%f %f %f %f", &c1.real, &c1.imag, &c2.real, &c2.imag) != 4) return 1;

    sum.real = c1.real + c2.real;
    sum.imag = c1.imag + c2.imag;

    prod.real = (c1.real * c2.real) - (c1.imag * c2.imag);
    prod.imag = (c1.real * c2.imag) + (c1.imag * c2.real);

    printf("Sum = %.1f + %.1fi\n", sum.real, sum.imag);
    printf("Product = %.1f + %.1fi\n", prod.real, prod.imag);
    return 0;
}
```

EXPLANATION & REASONING: Uses complex arithmetic formulas: - Addition: $(a+ib) + (c+id) = (a+c) + i(b+d)$ - Multiplication: $(a+ib)(c+id) = (ac-bd) + i(ad+bc)$.

STEP-BY-STEP DRY RUN: $c1 = 3 + 2i$, $c2 = 1 + 7i$. - Sum = $(3+1) + (2+7)i = 4 + 9i$. - Product real = $3*1 - 2*7 = 3 - 14 = -11$. - Product imag = $3*7 + 2*1 = 21 + 2 = 23$. Product = $-11 + 23i$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Problem 5.9: Car Structure Management

Design a structure 'Car' (id, model, rate per day) and calculate total rental cost for specified days.

```
#include <stdio.h>

struct Car {
    int id;
    char model[30];
    float rate_per_day;
};

int main() {
    struct Car car;
    int days = 5;
    if (scanf("%d %s %f", &car.id, car.model, &car.rate_per_day) != 3) return 1;

    float total = car.rate_per_day * days;
    printf("Car ID: %d, Model: %s, Total Rental for %d days = %.2f\n", car.id, car.model, days, total);
    return 0;
}
```

EXPLANATION & REASONING: Multiplies 'rate_per_day' member field of 'Car' structure by the specified duration 'days' to get total rental expense.

STEP-BY-STEP DRY RUN: $car = \{1, Sedan, 1500.0\}$, $days = 5$. $total = 1500.0 * 5 = 7500.00$.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

SECTION 6: Pointers

Problem 6.1: Basic Pointer Declaration

Demonstrate basic pointer variable declaration, address-of (`&`), and dereference (`\*`) operations.

```
#include <stdio.h>

int main() {
    int m = 10, n, o;
    int *z = &m;

    printf("z stores the address of m = %p\n", (void*)z);
    printf("**z stores the value of m = %d\n", *z);
    printf("&m is the address of m = %p\n", (void*)&m);
    printf("&n stores the address of n = %p\n", (void*)&n);
    printf("&o stores the address of o = %p\n", (void*)&o);
    printf("&z stores the address of z = %p\n", (void*)&z);
    return 0;
}
```

EXPLANATION & REASONING: `int *z = &m` initializes pointer variable `z` with the memory address of variable `m`. Dereferencing `*z` accesses the value stored at that memory location.

STEP-BY-STEP DRY RUN: `m = 10`. `z` points to `m`. `*z` resolves to 10. Printing `%p` displays hex address of `m`.

Time Complexity: O(1) Space Complexity: O(1)

Problem 6.2: Pointer Handling Demonstration

Demonstrate handling pointer reassignments and indirect variable mutations via pointer access.

```
#include <stdio.h>

int main() {
    int m = 29;
    int *ab = &m;
    printf("Address of m : %p Value of m : %d\n", (void*)&m, m);

    m = 34;
    printf("Content of pointer ab : %d\n", *ab);

    *ab = 7;
    printf("Value of m : %d\n", m);
    return 0;
}
```

EXPLANATION & REASONING: Since `ab` stores address of `m`, assigning `m = 34` changes value via direct assignment, and assigning `*ab = 7` mutates value via indirect pointer reference.

STEP-BY-STEP DRY RUN: 1. `m = 29`, `ab=&m`. 2. `m = 34` -> `*ab` reads 34. 3. `*ab = 7` -> `m` becomes 7.

Time Complexity: O(1) Space Complexity: O(1)

Problem 6.3: Use of & and * Operators

Demonstrate referencing addresses (&) and values (*) for int, float, and char variable types.

```
#include <stdio.h>

int main() {
    int m = 300;
    float fx = 300.600006f;
    char cht = 'z';

    printf("m = %d fx = %f cht = %c\n\n", m, fx, cht);
    printf("address of m = %p\naddress of fx = %p\naddress of cht = %p\n\n",
        (void*)&m, (void*)&fx, (void*)&cht);
    printf("value at address of m = %d\nvalue at address of fx = %f\nvalue at address of cht = %c\n",
        *(&m), *(&fx), *(&cht));
    return 0;
}
```

EXPLANATION & REASONING: &var obtains the memory location of var, while *(&var) cancels out to yield the underlying value stored at that location.

STEP-BY-STEP DRY RUN: *(&m) -> dereferences address of m -> 300. *(&fx) -> dereferences address of fx -> 300.600006. *(&cht) -> dereferences address of cht -> 'z'.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 6.4: Add Two Numbers with Pointers

Write a program in C to add two numbers using pointers.

```
#include <stdio.h>

int main() {
    int f = 5, s = 6, sum;
    int *p = &f, *q = &s;

    sum = *p + *q;
    printf("The sum of the entered numbers is : %d\n", sum);
    return 0;
}
```

EXPLANATION & REASONING: Dereferences integer pointers p and q (*p and *q) to fetch values 5 and 6, adding them together into variable sum.

STEP-BY-STEP DRY RUN: p = &f (val 5), q = &s (val 6). sum = *p + *q = 5 + 6 = 11.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 6.5: Add Numbers Using Call by Reference

Write a program in C to add numbers using call by reference.

```
#include <stdio.h>

void addNumbers(int *a, int *b, int *sum) {
    *sum = *a + *b;
}

int main() {
    int n1 = 5, n2 = 6, sum;
    addNumbers(&n1, &n2, &sum);
    printf("The sum of %d and %d is %d\n", n1, n2, sum);
    return 0;
}
```

EXPLANATION & REASONING: Passes addresses `&n1`, `&n2`, and `&sum` to function `addNumbers`. The function modifies `sum` directly in calling stack frame via pointer dereference.

STEP-BY-STEP DRY RUN: `addNumbers(&n1, &n2, &sum)` calculates `*sum = 5 + 6 = 11`. Back in `main()`, `sum` is 11.

Time Complexity: O(1)

Space Complexity: O(1)

Problem 6.6: Maximum of Two Using Pointer

Find the maximum number between two numbers using pointers.

```
#include <stdio.h>

int main() {
    int a = 5, b = 6;
    int *pa = &a, *pb = &b;

    if (*pa > *pb) {
        printf("%d is the maximum number.\n", *pa);
    } else {
        printf("%d is the maximum number.\n", *pb);
    }
    return 0;
}
```

EXPLANATION & REASONING: Compares dereferenced values `*pa` and `*pb` in condition `*pa > *pb` to determine maximum integer.

STEP-BY-STEP DRY RUN: `*pa = 5`, `*pb = 6`. Condition `5 > 6` is False. Else branch prints '6 is the maximum number.'

Time Complexity: O(1)

Space Complexity: O(1)

Problem 6.7: Print Array Using Pointer

Store N elements in an array and print the elements using pointer arithmetic.

```
#include <stdio.h>

int main() {
    int n = 5;
    int arr[] = {5, 7, 2, 9, 8};
    int *ptr = arr;

    printf("The elements you entered are :\n");
    for (int i = 0; i < n; i++) {
        printf("element - %d : %d\n", i, *(ptr + i));
    }
    return 0;
}
```

EXPLANATION & REASONING: Pointer arithmetic `*(ptr + i)` calculates the address offset `ptr + i` (stepping by `sizeof(int)` bytes) and dereferences it to fetch element `arr[i]`.

STEP-BY-STEP DRY RUN: `ptr = &arr[0]`. - `i=0`: `*(ptr+0) = 5` - `i=1`: `*(ptr+1) = 7` - `i=2`: `*(ptr+2) = 2` - `i=3`: `*(ptr+3) = 9` - `i=4`: `*(ptr+4) = 8`.

Time Complexity: $O(N)$

Space Complexity: $O(1)$

Problem 6.8: Permutations of a String Using Pointers

Print all permutations of a given string using pointers and backtracking recursion.

```
#include <stdio.h>
#include <string.h>

void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}

void permute(char *str, int l, int r) {
    if (l == r) {
        printf("%s ", str);
    } else {
        for (int i = l; i <= r; i++) {
            swap((str + l), (str + i));
            permute(str, l + 1, r);
            swap((str + l), (str + i)); // backtrack
        }
    }
}

int main() {
    char str[] = "abcd";
    printf("The permutations of the string are :\n");
    permute(str, 0, strlen(str) - 1);
    printf("\n");
    return 0;
}
```

EXPLANATION & REASONING: Recursively swaps character pointers `(str + l)` and `(str + i)` to generate $N!$ unique orderings of the N -character string.

STEP-BY-STEP DRY RUN: For "abcd" ($N=4$), generates $4! = 24$ permutations starting with "abcd" and backtracking through swap operations.

Time Complexity: $O(N * N!)$

Space Complexity: $O(N)$ recursion stack space

Problem 6.9: Largest Element Using Dynamic Memory Allocation

Find the largest element among N float numbers allocated dynamically.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    float *arr = (float *)malloc(n * sizeof(float));
    if (arr == NULL) return 1;

    arr[0] = 5.0f; arr[1] = 7.0f; arr[2] = 2.0f; arr[3] = 9.0f; arr[4] = 8.0f;

    float max = *arr;
    for (int i = 1; i < n; i++) {
        if (*(arr + i) > max) {
            max = *(arr + i);
        }
    }
    printf("The Largest element is : %.2f\n", max);
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Allocates float array on heap memory using `malloc()`, traverses it using pointer offset `*(arr + i)` to locate maximum float value, and releases heap memory via `free()`.

STEP-BY-STEP DRY RUN: arr = [5.0, 7.0, 2.0, 9.0, 8.0]. Max initialized to 5.0. Traverses array -> updates max to 7.0 -> 9.0. Final max = 9.00.

Time Complexity: $O(N)$

Space Complexity: $O(N)$ heap allocation

Problem 6.10: String Length Using Pointer

Calculate the length of a string using pointer increment.

```
#include <stdio.h>

int main() {
    char str[] = "w3resource";
    char *ptr = str;
    int len = 0;

    while (*ptr != '\0') {
        len++;
        ptr++;
    }
    printf("The length of the given string %s is : %d\n", str, len);
    return 0;
}
```

EXPLANATION & REASONING: Increments pointer `ptr` and length counter `len` until dereferenced pointer `*ptr` hits null character `\0`.

STEP-BY-STEP DRY RUN: str = "w3resource". Pointer advances 10 positions through characters 'w','3','r','e','s','o','u','r','c','e'. Stops at `\0`. Output len = 10.

Time Complexity: $O(N)$

Space Complexity: $O(1)$

SECTION 7: Dynamic Memory Allocation

Problem 7.1: Store N Integers

Allocate memory using `malloc()` for N integers, input and display them.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
    }
    printf("Stored Integers: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: `malloc(n \* sizeof(int))` requests `n \* 4` bytes from heap memory. Returning pointer is checked against `NULL` to verify successful allocation.

STEP-BY-STEP DRY RUN: n=5. Heap allocated for 5 ints. Loop assigns arr[0]=10, arr[1]=20, arr[2]=30, arr[3]=40, arr[4]=50. Prints array, then frees memory.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 7.2: Find Sum of Array

Dynamically allocate an array, compute and print the sum of all elements.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5, sum = 0;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) {
        arr[i] = i + 1;
        sum += arr[i];
    }
    printf("Sum = %d\n", sum);
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Allocates dynamic integer array, populates values, and sums them in a single pass before freeing heap memory.

STEP-BY-STEP DRY RUN: arr = [1, 2, 3, 4, 5]. sum = 1 + 2 + 3 + 4 + 5 = 15.

Time Complexity: O(N)

Space Complexity: O(N)

Problem 7.3: Find Maximum Element

Allocate memory for N integers and find the largest number.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 4;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    arr[0] = 12; arr[1] = 45; arr[2] = 23; arr[3] = 9;

    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
    }
    printf("Max = %d\n", max);
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Dynamically creates integer buffer, initializes max with `arr[0]`, updates max on finding larger element during iteration.

STEP-BY-STEP DRY RUN: arr = [12, 45, 23, 9]. Max starts at 12. i=1: 45 > 12 -> max = 45. i=2: 23 < 45. i=3: 9 < 45. Final max = 45.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 7.4: Reverse an Array

Dynamically allocate an array and print elements in reverse order.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) arr[i] = i + 1;

    printf("Reversed Array: ");
    for (int i = n - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Allocates array dynamically, fills with 1..N, and loops backward from `n - 1` down to 0 for reverse rendering.

STEP-BY-STEP DRY RUN: arr = [1, 2, 3, 4, 5]. Print loop starting index 4 down to 0 -> Outputs '5 4 3 2 1 '.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 7.5: Average of Numbers

Store N floating-point numbers using `malloc()` and find their average.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 4;
    float *arr = (float *)malloc(n * sizeof(float));
    if (arr == NULL) return 1;

    arr[0] = 10.5f; arr[1] = 20.5f; arr[2] = 30.0f; arr[3] = 40.0f;
    float sum = 0.0f;

    for (int i = 0; i < n; i++) sum += arr[i];

    printf("Average = %.2f\n", sum / n);
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Allocates heap memory for floats, sums array elements, and divides total sum by count `n`.

STEP-BY-STEP DRY RUN: arr = [10.5, 20.5, 30.0, 40.0]. sum = 101.0. avg = 101.0 / 4 = 25.25.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Problem 7.6: Resize an Array

Allocate memory for N integers, increase size to M using `realloc()`, and print final array.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 3, m = 5;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) arr[i] = i + 1;

    arr = (int *)realloc(arr, m * sizeof(int));
    if (arr == NULL) return 1;

    arr[3] = 4;
    arr[4] = 5;

    for (int i = 0; i < m; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: `realloc()` re-allocates contiguous dynamic memory block preserving existing data. Expands buffer from size 3 to 5.

STEP-BY-STEP DRY RUN: 1. malloc 3 ints -> [1, 2, 3]. 2. realloc to 5 ints -> [1, 2, 3, _, _]. 3. Assign arr[3]=4, arr[4]=5 -> [1, 2, 3, 4, 5].

Time Complexity: $O(M)$

Space Complexity: $O(M)$

Problem 7.7: Insert an Element

Dynamically allocate an array and insert an element at a given index position.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 4, pos = 2, val = 99;
    int *arr = (int *)malloc((n + 1) * sizeof(int));
    if (arr == NULL) return 1;

    arr[0] = 10; arr[1] = 20; arr[2] = 30; arr[3] = 40;

    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = val;

    for (int i = 0; i <= n; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Shifts elements starting from end index rightward by one step to open a slot at target position `pos`, then inserts `val`.

STEP-BY-STEP DRY RUN: arr = [10, 20, 30, 40, _]. - Shift i=4: arr[4]=arr[3]=40. - Shift i=3: arr[3]=arr[2]=30. - Insert pos=2: arr[2] = 99. Result: [10, 20, 99, 30, 40].

Time Complexity: O(N)

Space Complexity: O(N)

Problem 7.8: Delete an Element

Remove an element from a given position in a dynamically allocated array.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 4, pos = 1;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    arr[0] = 10; arr[1] = 20; arr[2] = 30; arr[3] = 40;

    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }

    for (int i = 0; i < n - 1; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
```

EXPLANATION & REASONING: Overwrites element at target index `pos` by shifting all remaining rightward elements left by one position.

STEP-BY-STEP DRY RUN: arr = [10, 20, 30, 40], pos=1 (value 20). - i=1: arr[1] = arr[2] = 30. - i=2: arr[2] = arr[3] = 40. Result length = 3 -> [10, 30, 40].

Time Complexity: O(N)

Space Complexity: O(N)

Problem 7.9: Matrix Addition

Dynamically allocate two 2D matrices, perform addition, and display the result matrix.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int r = 2, c = 2;
    int **a = (int **)malloc(r * sizeof(int *));
    int **b = (int **)malloc(r * sizeof(int *));
    int **res = (int **)malloc(r * sizeof(int *));

    for (int i = 0; i < r; i++) {
        a[i] = (int *)malloc(c * sizeof(int));
        b[i] = (int *)malloc(c * sizeof(int));
        res[i] = (int *)malloc(c * sizeof(int));
    }

    a[0][0] = 1; a[0][1] = 2; a[1][0] = 3; a[1][1] = 4;
    b[0][0] = 5; b[0][1] = 6; b[1][0] = 7; b[1][1] = 8;

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            res[i][j] = a[i][j] + b[i][j];
            printf("%d ", res[i][j]);
        }
        printf("\n");
    }

    for (int i = 0; i < r; i++) {
        free(a[i]); free(b[i]); free(res[i]);
    }
    free(a); free(b); free(res);
    return 0;
}
```

EXPLANATION & REASONING: Allocates array of pointers (`int**`) representing rows, then allocates column memory per row pointer. Adds corresponding cell entries `res[i][j] = a[i][j] + b[i][j]`.

STEP-BY-STEP DRY RUN: A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]]. - `res[0][0] = 1+5 = 6`. - `res[0][1] = 2+6 = 8`. - `res[1][0] = 3+7 = 10`. - `res[1][1] = 4+8 = 12`.

Time Complexity: $O(R * C)$

Space Complexity: $O(R * C)$

Problem 7.10: String Copy

Allocate memory for a string using `malloc()` and copy string without using `strcpy()`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char src[] = "Dynamic Memory Allocation";
    int len = 0;
    while (src[len] != '\0') len++;

    char *dest = (char *)malloc((len + 1) * sizeof(char));
    if (dest == NULL) return 1;

    for (int i = 0; i <= len; i++) {
        dest[i] = src[i];
    }
    printf("Copied String: %s\n", dest);
    free(dest);
    return 0;
}
```

EXPLANATION & REASONING: Calculates string length, allocates `len + 1` bytes on heap for string plus null character, and copies characters in loop.

STEP-BY-STEP DRY RUN: src len = 25. malloc allocates 26 bytes. Loop copies characters 0..25 including `'\0'`. Prints destination string.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

SECTION 8: Company Specific Questions

Problem 8.1: Maximum Sum Subarray with Equal First and Last Element

Given an array of N integers, find the maximum sum of a contiguous subarray whose first and last elements are equal. If no such subarray exists (except single element), return maximum element in the array.

```
#include <stdio.h>

int main() {
    int N = 8;
    int arr[] = {3, 2, 4, 5, 2, 6, 3, 1};
    long long maxSum = -1e18;
    int found = 0;

    for (int i = 0; i < N; i++) {
        long long currentSum = 0;
        for (int j = i; j < N; j++) {
            currentSum += arr[j];
            if (i != j && arr[i] == arr[j]) {
                if (currentSum > maxSum) maxSum = currentSum;
                found = 1;
            }
        }
    }

    if (!found) {
        maxSum = arr[0];
        for (int i = 1; i < N; i++) {
            if (arr[i] > maxSum) maxSum = arr[i];
        }
    }

    printf("Output: %lld\n", maxSum);
    return 0;
}
```

EXPLANATION & REASONING: Traverses all contiguous subarrays `arr[i...j]`. If start element `arr[i]` equals end element `arr[j]` with `i != j`, checks if subarray sum is maximum. Fallback returns max element if no matching pairs exist.

STEP-BY-STEP DRY RUN: `arr = [3, 2, 4, 5, 2, 6, 3, 1]`. - Subarray starting at index 0 (val 3) and ending at index 6 (val 3): `[3, 2, 4, 5, 2, 6, 3]`. - First and last elements match (`3 == 3`). - Sum = `3 + 2 + 4 + 5 + 2 + 6 + 3 = 25`. - Maximum sum among equal-boundary subarrays = 25.

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

Problem 8.2: Longest Substring Without Repeating Characters

Given a string consisting of lowercase English letters, find the length of the longest substring containing no repeated characters.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "abcabcbb";
    int n = strlen(s);
    int lastPos[256];
    for (int i = 0; i < 256; i++) lastPos[i] = -1;

    int maxLen = 0, start = 0;
    for (int i = 0; i < n; i++) {
        if (lastPos[(unsigned char)s[i]] >= start) {
            start = lastPos[(unsigned char)s[i]] + 1;
        }
        lastPos[(unsigned char)s[i]] = i;
        int len = i - start + 1;
        if (len > maxLen) maxLen = len;
    }

    printf("Output: %d\n", maxLen);
    return 0;
}
```

EXPLANATION & REASONING: Sliding window technique maintaining left index `start`. `lastPos` array records last seen index of each character. When a repeat is encountered within current window (`lastPos[char] >= start`), `start` jumps past duplicate.

STEP-BY-STEP DRY RUN: s = "abcabcbb". - i=0('a'): start=0, len=1, max=1. - i=1('b'): start=0, len=2, max=2. - i=2('c'): start=0, len=3, max=3. - i=3('a'): lastPos['a']=0 >= start(0) -> start=1 ('b'). len=3-1+1=3. - i=4('b'): start jumps to 2 ('c'). len=3. - i=5('c'): start jumps to 3. len=3. - Max length observed = 3 (substring "abc").

Time Complexity: O(N)

Space Complexity: O(1) - fixed 256 size character map